HOMEWORK 2

1.Define an integer array. Declare an integer pointer and initialise it to first element of array. Use pointer arithmetic to traverse the array and print the elements.

CODE:

```
#include <stdio.h>
int main(){
int arr[10] ={10,20,30,40,50,60,70,80,90,100};
int *ptr;
ptr = arr;
printf("The elements in the array are:\n");
for(int i = 0; i < 10; i++){
   printf("%d ",*(ptr+i));
}
return 0;
}
```

OUTPUT:

```
The elements in the array are:
10 20 30 40 50 60 70 80 90 100
```

2.Calculate string length using pointers.

CODE:

```
#include <stdio.h>
int main(){
   char str[200];
   char *pstr ;
    pstr= str;
    int count=0;

    printf("Enter the string: ");
    fgets(str,sizeof(str),stdin);

    for(int i = 0; *(pstr+i)  != '\0'; i++){
      if(*(pstr+i) != '\n'){
      count++;
      }
   }
   printf("Number of strings: %d",count);
    return 0;
}
```

OUTPUT:

```
Enter the string: hello world
Number of strings: 11
```

3.Write a function that calculates both the sum and product of two numbers. Return these two results using pointers passed as parameters to the function.

CODE:

```
#include <stdio.h>


void perform(int a, int b,int *sptr, int *mptr){
   *sptr = a+b;
   *mptr = a*b;
}


int main(){
   int num1, num2;
   int sum,product;


   printf("Enter two numbers: ");
   scanf("%d%d",&num1,&num2);


   perform(num1,num2,&sum,&product);


   printf("Sum: %d\nProduct: %d",sum, product);


   return 0;
```

```
}
```

OUTPUT:

```
Enter two numbers: 12
5
Sum: 17
Product: 60
```

4.Create an array of structures. Use dynamic memory allocation to allocate memory space and use a structure pointer with pointer arithmetic to iterate and access/modify members.

CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct students{
   int r_no;
   char name[20];
   float avg_mark;
};

void add(struct students *ptr, int n){
    for(int i=0; i<n; i++){
      (ptr+i)->avg_mark +=5;
```

```
    }
}

int main(){
    struct students *ptr;
    int n,i;

    printf("Enter number of students: ");
    scanf("%d",&n);

    ptr = (struct students*) malloc (n * sizeof(struct students));

    if(ptr==NULL){
        printf("Memory allocation failed");
        return 1;
    }

    printf("Enter details:\n");
    for(i=0; i<n; i++){
        printf("Roll number: ");
        scanf("%d",&(ptr+i)->r_no);
        printf("Name: ");
        scanf("%s",&(ptr+i)->name);
        printf("Average mark: ");
        scanf("%f",&(ptr+i)->avg_mark);
```

```
    }

    add(ptr, n);

    printf("\nAdditional 5 marks\n");

    for(i=0; i<n; i++){
        printf("\nRoll number: %d",(ptr+i)->r_no);
        printf("\nName: %s",(ptr+i)->name);
        printf("\nAverage mark: %f",(ptr+i)->avg_mark);
        printf("\n");

    }

    return 0;
}
```

OUTPUT:

```
Enter number of students: 2
Enter details:
Roll number: 5028
Name: Sangavi
Average mark: 88.5
Roll number: 5116
Name: Pavatharani
Average mark: 82.4
```

Additional 5 marks

Roll number: 5028

Name: Sangavi

Average mark: 93.500000

Roll number: 5116

Name: Pavatharani

Average mark: 87.400002

5.Define two functions with same signature. Declare a function pointer and use the pointer to call both functions.

CODE:

```
#include <stdio.h>

int add(int a, int b){
   int sum = a+b;
   return sum;
}

int mul(int a, int b){
   int product = a*b;
```

```
    return product;
}

int main(){
    int num1, num2;
    int (*ptr)(int, int);

    printf("Enter two numbers: ");
    scanf("%d%d",&num1,&num2);

    ptr = add;
    printf("Sum: %d",ptr(num1,num2));

    ptr = mul;
    printf("\nProduct: %d",ptr(num1,num2));

    return 0;
}
```

OUTPUT:

```
Enter two numbers: 12
5
Sum: 17
Product: 60
```

6.Declare an integer, a pointer to it and a double pointer. Print the values and addresses at each level.

CODE:

```
#include <stdio.h>


int main(){
   int num = 10;
   int *ptr = &num;
   int **dptr = &ptr;


   printf("Value of num: %d",num);
   printf("\nAddress of num: %p",(void *)&num);
   printf("\nValue stored in ptr: %p",(void *)ptr);
   printf("\nValue obtained using *ptr: %d",*ptr);
   printf("\nAddress of ptr: %p",(void *)&ptr);
   printf("\nValue stored in dptr: %p",(void *)dptr);
   printf("\nValue obtained using *dptr: %p",(void *)*dptr);
   printf("\nValue obtained using **dptr: %d",**dptr);
   printf("\nAddress of dptr: %p",(void*)&dptr);


   return 0;


}
```

OUTPUT:

Value of num: 10

Address of num: 0061FF1C

Value stored in ptr: 0061FF1C

Value obtained using *ptr: 10

Address of ptr: 0061FF18

Value stored in dptr: 0061FF18

Value obtained using *dptr: 0061FF1C

Value obtained using **dptr: 10

Address of dptr: 0061FF1E

7.Sort an array of floating point numbers using selection sort.

CODE:

```
#include <stdio.h>


int main(){
   int n,i,j,min;
   float arr[100],temp;
   printf("Enter the size of the array: ");
   scanf("%d", &n);


   printf("Enter the  floating numbers:\n");
   for(i=0; i<n; i++){
      scanf("%f",&arr[i]);
```

```
    }

    for(i=0; i<n-1; i++){
        min =i;
        for(j=i+1;j<n; j++){
        if(arr[j]<arr[min]){
            min = j;
        }
    }
        temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }

    printf("Sorted array:\n");
    for(i=0; i<n; i++){
        printf("%.2f ",arr[i]);
    }

    return 0;

}
```

OUTPUT:

```
Enter the size of the array: 5
Enter the  floating numbers:
```

1.3

98.7

45.7

3.4

76.3

Sorted array:

1.30 3.40 45.70 76.30 98.70

8.Define a structure student ,student with id, name and grade, create an array of students structure and sort them based on different field.

CODE:

```
#include <stdio.h>
#include <string.h>

struct student {
   int id;
   char name[50];
   float grade;
};

int main() {
   struct student s[100], temp;
   int n, i, j, choice;
```

```
printf("Enter number of students: ");
scanf("%d",&n);

printf("Enter details of students:\n");

for (i = 0; i < n; i++) {
   printf("\nStudent %d\n", i + 1);

   printf("Enter ID: ");
   scanf("%d", &s[i].id);

   printf("Enter name: ");
   scanf("%s", s[i].name);

   printf("Enter grade: ");
   scanf("%f", &s[i].grade);
}

printf("\nSort based on:\n");
printf("1. ID\n");
printf("2. Name\n");
printf("3. Grade\n");
printf("Enter your choice: ");
scanf("%d", &choice);
```

```
for (i = 0; i < n-1; i++) {
   for (j = i + 1; j < n; j++) {

      int condition = 0;

      if (choice == 1 && s[i].id > s[j].id)
         condition = 1;

      else if (choice == 2 && strcmp(s[i].name, s[j].name) > 0)
         condition = 1;

      else if (choice == 3 && s[i].grade > s[j].grade)
         condition = 1;

      if (condition) {
         temp = s[i];
         s[i] = s[j];
         s[j] = temp;
      }
   }
}

printf("\nStudents after sorting:\n");
```

```
    for (i = 0; i < n; i++) {
        printf("ID: %d  Name: %s  Grade: %.2f\n",
            s[i].id, s[i].name, s[i].grade);
    }

    return 0;
}
```

OUTPUT:

```
Enter number of students: 2
Enter details of students:

Student 1
Enter ID: 5028
Enter name: Sangavi
Enter grade: 9.8

Student 2
Enter ID: 5002
Enter name: Pavatharani
Enter grade: 9.6

Sort based on:
1. ID
2. Name
```

3. Grade

Enter your choice: 2

Students after sorting:

ID: 5002 Name: Pavatharani Grade: 9.60

ID: 5028 Name: Sangavi Grade: 9.80

9.Sort a set of negative numbers using radix sort.

CODE:

```
#include <stdio.h>
#define MAX 100

void radixSortpositive(int arr[], int n){
   int output[MAX];
   int count[10];
   int max = arr[0];

   for(int i=1; i<n; i++){
      if(arr[i] > max){
         max = arr[i];
      }
   }

   for(int exp=1; max/exp >0; exp*=10){
```

```
        for(int i=0; i<10; i++){
        count[i] = 0;
    }

        for(int i=0; i<n; i++){
        int digit = (arr[i] / exp) % 10;
        count[digit]++;
        }

    for(int i=1; i<10; i++){
        count[i] += count[i-1];
    }

    for(int i=n-1; i>=0; i--){
        int digit = (arr[i] / exp) % 10;
        output[count[digit]-1] = arr[i];
        count[digit]--;
    }

    for(int i=0; i<n; i++){
        arr[i] = output[i];
    }
  }
}
```

```
void radixSort(int arr[], int n){
    int positive[MAX], negative[MAX];
    int pos=0, neg=0;

  for(int i=0; i<n; i++){
    if(arr[i] < 0){
     negative[neg++] = -arr[i];
    }
    else{
     positive[pos++] = arr[i];
    }
  }

  if(pos > 0){
    radixSortpositive(positive, pos);
  }

  if(neg > 0){
    radixSortpositive(negative, neg);
  }

  int k=0;

  for(int i=neg-1; i>=0; i--){
```

```
        arr[k++] = -negative[i];
    }

    for(int i=0; i<pos; i++){
        arr[k++] = positive[i];
    }
}

int main(){
    int n;
    printf("Enter the size of array: ");
    scanf("%d",&n);

    int arr[n];

    printf("Enter the elements: ");
    for(int i=0; i<n; i++){
        scanf("%d",&arr[i]);
    }

    radixSort(arr, n);
    printf("Sorted array:\n");
    for(int i=0; i<n; i++){
        printf("%d ",arr[i]);
    }
```

```
return 0;
}
```

OUTPUT:

```
Enter the size of array: 8
Enter the elements: 10
-3
7
-25
100
-50
8
-2
Sorted array:
-50 -25 -3 -2 7 8 10 100
```

10.Write a C program to copy one array to another using pointers.

CODE:

```
#include <stdio.h>

int main(){
   int i,n;
   int arr1[100];
   int copy[100];
```

```
    int *ptr = arr1;

    printf("Enter the size of the array: ");
    scanf("%d",&n);

    printf("Enter the elements:\n");
    for(i=0; i<n; i++){
        scanf("%d",&arr1[i]);
    }

    for(i=0; i<n; i++){
        copy[i] = *(ptr+i);
    }

    printf("Copied array:\n");
    for(i=0; i<n; i++){
        printf("%d ",copy[i]);
    }

    return 0;
}
```

OUTPUT:

```
Enter the size of the array: 5
Enter the elements:
12
```

34

56

78

89

Copied array:

12 34 56 78 89